Porównawcza ocena modeli klasyfikacyjnych

Ocena wpływu strategii walidacji — walidacji krzyżowej, bootstrapu i podziału na zbiór treningowy i testowy — na zbiorze Vehicle z pakietu mlbench

Bartosz Golubiewski, Kajetan Marszałek

2026-10-24

Spis treści

1 Wstęp **2**

2 Opis danych **3**

2.1 Rozkład klas 4

2.2 Zmienność atrybutów 4

2.3 Zdolność dyskryminacyjna 4

2.4 Przygotowanie danych 4

3 Metoda k-najbliższych sąsiadów (*k-Nearest Neighbors*) **7**

3.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test) 7

3.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*) 9

3.3 Wnioski — metoda k-najbliższych sąsiadów 11

4 Drzewa klasyfikacyjne (*classification trees*) **12**

4.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test). 12

4.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*) 15

4.3 Wnioski — drzewa klasyfikacyjne 15

5 Klasyfikator Naiwny Bayesa (*Naive Bayes*) 16

5.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test) 16

5.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*) . . 17

5.3 Wnioski - Klasyfikator Naiwny Bayesa 18

6 Algorytm *Bagging* 18

6.1 Optymalizacja parametru `nbagg` 18

6.2 Analiza błędu klasyfikacji 20

7 Algorytm *AdaBoost* 20

7.1 Optymalizacja parametru `mfinal` 22

7.2 Ranking ważności cech w algorytmie *AdaBoost* 22

7.3 Analiza błędu klasyfikacji 23

8 Lasy losowe 23

8.1 Optymalizacja parametru `ntree` 23

8.2 Ranking ważności cech w algorytmie *Random Forest* 25

8.3 Analiza błędu klasyfikacji 26

8.4 Porównanie metod zespołowych z pojedynczym drzewem decyzyjnym 26

9 Metoda wektorów nośnych (SVM) 27

9.1 Optymalizacja parametrów `C` oraz `Kernel` 27

9.2 Optymalizacja parametru `gamma` dla jądra radialnego 29

9.3 Analiza błędu klasyfikacji 31

10 Podsumowanie wyników 32

1 Wstęp

Celem projektu jest porównanie wybranych modeli uczenia nadzorowanego oraz ocena wpływu zastosowanej metody walidacji na uzyskiwane wyniki. Analiza obejmuje walidację krzyżową (*Cross-Validation*), metodę *Bootstrap* oraz pojedynczy podział danych na zbiór treningowy i testowy. Badania przeprowadzono na zbiorze **Vehicle** z pakietu `mlbench`, który zawiera dane dotyczące sylwetek pojazdów. Porównanie pozwala ocenić skuteczność poszczególnych modeli oraz przydatność analizowanych strategii walidacji w ocenie jakości klasyfikacji.

2 Opis danych

W niniejszym projekcie użyto zbioru danych **Vehicle** z pakietu `mlbench`. Jest to pomniejszona wersja zbioru stworzonego w latach 1986–87 przez Barry'ego Shepherda oraz Pete'a Mowfortha. Szczegółowe dotyczące danych można znaleźć na stronie repozytorium UCI — *Statlog (Vehicle Silhouettes) Dataset*.

Zbiór zawiera informacje dotyczące **846** pojazdów, podzielone na **19** atrybutów. Wśród nich 18 zmiennych (typu `numeric`) opisujących geometryczne cechy pojazdów:

- `Comp` (*compactness*) — zwartość obiektu
- `Circ`(*circularity*) — kolistość
- `D.Circ` (*distance circularity*) — kolistość oparta na odległości od krawędzi
- `Rad.Ra` (*radius ratio*) — stosunek promieni
- `Pr.Axis.Ra` (*principal axis aspect ratio*) — proporcja osi głównych
- `Max.L.Ra` (*maximum length aspect ratio*) — proporcja maksymalnej długości
- `Scat.Ra` (*scatter ratio*) — stosunek rozproszenia
- `Elong` (*elongatedness*) — wydłużenie
- `Pr.Axis.Rect` (*principal axis rectangularity*) — prostokątność względem osi głównych
- `Max.L.Rect` (*maximum length rectangularity*) — prostokątność względem maksymalnej długości
- `Sc.Var.Maxis` (*scaled variance along major axis*) — znormalizowana wariancja względem osi głównej
- `Sc.Var.maxis` (*scaled variance along minor axis*) — znormalizowana wariancja względem osi pomocniczej
- `Ra.Gyr` (*scaled radius of gyration*) — znormalizowany promień bezwładności
- `Skew.Maxis` (*skewness about major axis*) — skośność względem osi głównej
- `Skew.maxis` (*skewness about minor axis*) — skośność względem osi pomocniczej
- `Kurt.maxis` (*kurtosis about minor axis*) — kurtoza względem osi pomocniczej
- `Kurt.Maxis` (*kurtosis about major axis*) — kurtoza względem osi głównej
- `Holl.Ra` (*hollows ratio*) — współczynnik pustych przestrzeni

oraz zmienną `Class` (typu `factor`) opisującą rodzaj pojazdu (`OPEL`, `SAAB`, `BUS`, `VAN`). Jest to zmienna zawierająca etkietki klas.

Zbiór nie zawiera braków danych.

2.1 Rozkład klas

Tabela 1: Udział procentowy danej klasy w danych

Klasa	Udział w danych
bus	25.77
opel	25.06
saab	25.65
van	23.52

W tabeli 1 przedstawiono procentowy udział poszczególnych klas pojazdów w analizowanym zbiorze danych. Można zauważyć, że rozkład klas jest stosunkowo równomierny — każda z klas stanowi około 25% wszystkich obserwacji. Jedynie klasa `Van` występuje nieznacznie rzadziej od pozostałych.

Gdyby wszystkie obserwacje zostały przypisane wyłącznie do najczęściej występującej klasy (`Bus`), błąd klasyfikacji, rozumiany jako odsetek błędnie sklasyfikowanych obiektów, wyniósłby około 0.74.

2.2 Zmienność atrybutów

Na rysunku 1 przedstawiono wykresy pudełkowe zmiennych ilościowych. Widać wyraźne różnice w ich skali oraz zróżnicowaniu. Na przykład `Sc.Var.maxis` przyjmuje wartości z przedziału 184–1018, natomiast `Skew.maxis` obejmuje wartości od 0 do 22. Na wykresach widoczne są również obserwacje odstające.

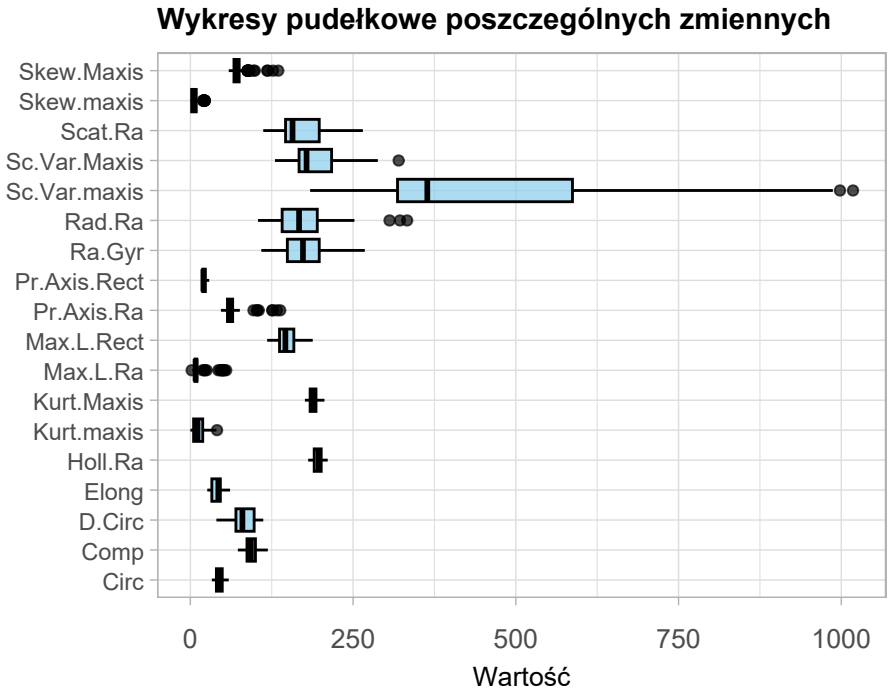
Tak duże różnice w rozrzucie cech wskazują na potrzebę standaryzacji przed zastosowaniem algorytmów klasyfikacyjnych, które są wrażliwe na skalę zmiennych.

2.3 Zdolność dyskryminacyjna

Na podstawie analizy estymatorów gęstości można stwierdzić, że u wszystkich zmiennych obserwuje się silne nakładanie rozkładów. Jednak niektóre zmienne wykazują lepsze zdolności dyskryminacyjne. W szczególności są to cechy `Scat.Ra`, `Elong` oraz `D.Circ`, których wykresy estymatorów gęstości przedstawiono na rysunku 2. Zmienne te pozwalają na częściowe rozdzielenie klas pojazdów.

2.4 Przygotowanie danych

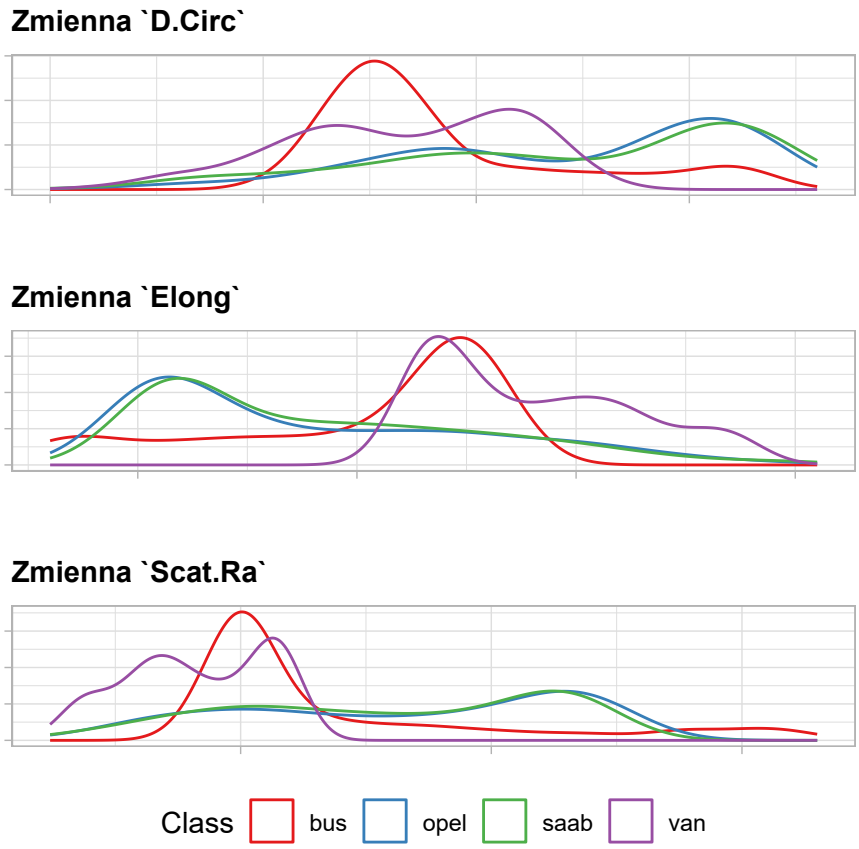
W celu budowy i oceny modeli klasyfikacyjnych zbiór danych podzielono na część uczącą, obejmującą $\frac{2}{3}$ obserwacji, oraz część testową, zawierającą pozostałą $\frac{1}{3}$ danych.



Rysunek 1: Wykresy pudełkowe zmiennych ilościowych

Zgodnie z wnioskami przedstawionymi w rozdziale 2.2, przeprowadzono standaryzację zmiennych. W tym celu na podstawie zbioru uczącego zostały wyznaczone wartości średniej oraz odchylenia standardowego. Uzyskane parametry zastosowano do przeskalowania zarówno zbioru uczącego, jak i testowego.
Na potrzeby bardziej zaawansowanych schematów oceny dokładności (*Cross-validation*, *Bootstrap*) przygotowano również globalnie przeskalowaną wersję pełnego zbioru.

Wszystkie operacje losowe związane z podziałem danych przeprowadzono przy użyciu ustalonego ziarna generatora liczb równego `87124671`.



Rysunek 2: Estymatory gęstość wybranych zmiennych w zależności od zmiennej `Class`

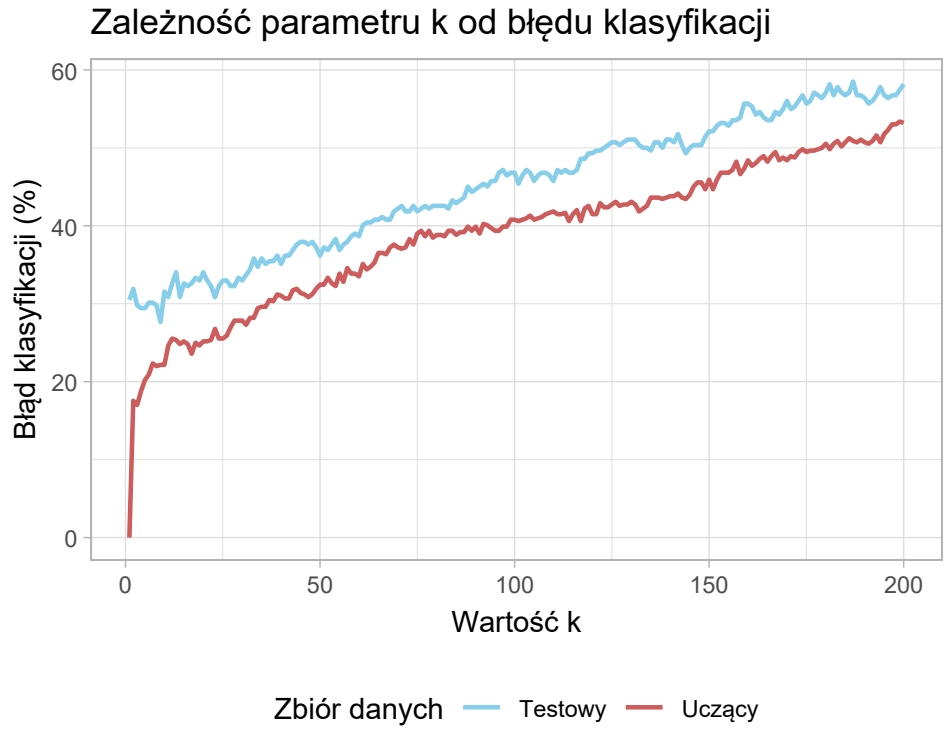
3 Metoda k-najbliższych sąsiadów (*k-Nearest Neighbors*)

3.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test)

3.1.1 Metoda k-najbliższych sąsiadów dla pełnego zbioru zmiennych.

Jako pierwszą zastosowano metodę k-najbliższych sąsiadów na pełnym zestawie atrybutów. W celu znalezienia optymalnej liczby sąsiadów przetestowano wartości parametru `k` od 1 do 200, wyznaczając błąd klasyfikacji zarówno na zbiorze uczącym, jak i testowym.

W obliczeniach użyto funkcji `knn()` z pakietu `class`.



Rysunek 3: Wykres zależności błędu klasyfikacji w zależności od liczby szukanych sąsiadów (k). Model zbudowany na podstawie wszystkich zmiennych.

Na rysunku 3 przedstawiono zależność błędu klasyfikacji metody od wartości parametru `k`, oznaczającego liczbę uwzględnianych sąsiadów. Procent niepoprawnie przypisanych obserwacji rośnie wraz z wartością parametru `k`. Błąd na zbiorze testowym jest większy od błędu zbioru uczącego. Warto zwrócić uwagę na błąd na zbiorze uczącym dla $k = 1$, który wynosi 0 — model idealnie zapamiętał dane treningowe, ale gorzej radzi sobie z nowymi przykładami.

Najlepsze wyniki na zbiorze testowym otrzymano dla `k` = 9, dla którego odsetek niepoprawnie przypisanych obserwacji wyniósł 27.66%.

Wizualizację macierzy pomyłek dla optymalnego parametru (`k` = 9) przedstawiono w tabelach 2 oraz 3. Wartości odsetka błędu wyniosły odpowiednio 22% dla zbioru uczącego oraz 28% dla zbioru testowego.

Tabela 2: Macierz pomyłek dla zbioru uczącego (pełne zmienne) Wiersze — klasy wyznaczone, kolumny — klasy rzeczywiste.

	bus	opel	saab	van
bus	141	2	8	5
opel	0	79	30	1
saab	1	50	94	0
van	3	10	10	130

Tabela 3: Macierz pomyłek dla zbioru testowego (pełne zmienne). Wiersze — klasy wyznaczone, kolumny — klasy rzeczywiste.

	bus	opel	saab	van
bus	70	1	3	8
opel	0	37	21	1
saab	0	25	42	0
van	3	8	9	54

Głównym źródłem błędów klasyfikatora są zbliżone cechy geometryczne samochodów osobowych klasy `Opel` oraz `Saab`. W zbiorze testowym aż 21 pojazdów marki Saab zostało błędnie uznane za Ople, z kolei 25 Opli sklasyfikowano jako Saaby.

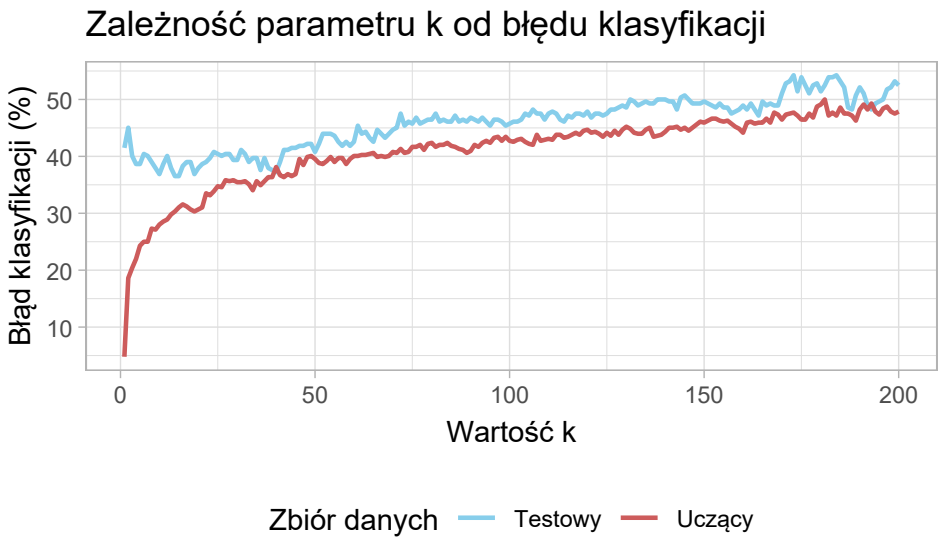
Natomiast model dobrze radzi sobie z rozpoznawaniem klas `Bus` oraz `Van`, które ze względu na swoje unikalne kształty są niemal bezbłędnie rozróżniane od reszty.

3.1.2 Metoda k-najbliższych sąsiadów dla zmiennych o najlepszej zdolności dyskryminacyjnej

Tę samą procedurę powtórzono, ograniczając zmienne wyłącznie do tych o dobrej zdolności dyskryminacyjnej — `Scat.Ra`, `Elong` oraz `D.Circ` (rodział 2.3).

Dla modelu o ograniczonej liczbie cech (Rysunek 4), minimum błędu na zbiorze testowym osiągnięto dla `k` = 14 z wynikiem 36.52%.

Tabela 4 prezentuje macierz pomyłek dla zredukowanego podzbioru atrybutów.



Rysunek 4: Wykres zależności błędu klasyfikacji w zależności od liczby szukanych sąsiadów (k). Model zbudowany na podstawie wybranych zmiennych.

Tabela 4: Macierz pomyłek dla zbioru testowego (wybrane 3 cechy). Wiersze — klasy wyznaczone, kolumny — klasy rzeczywiste.

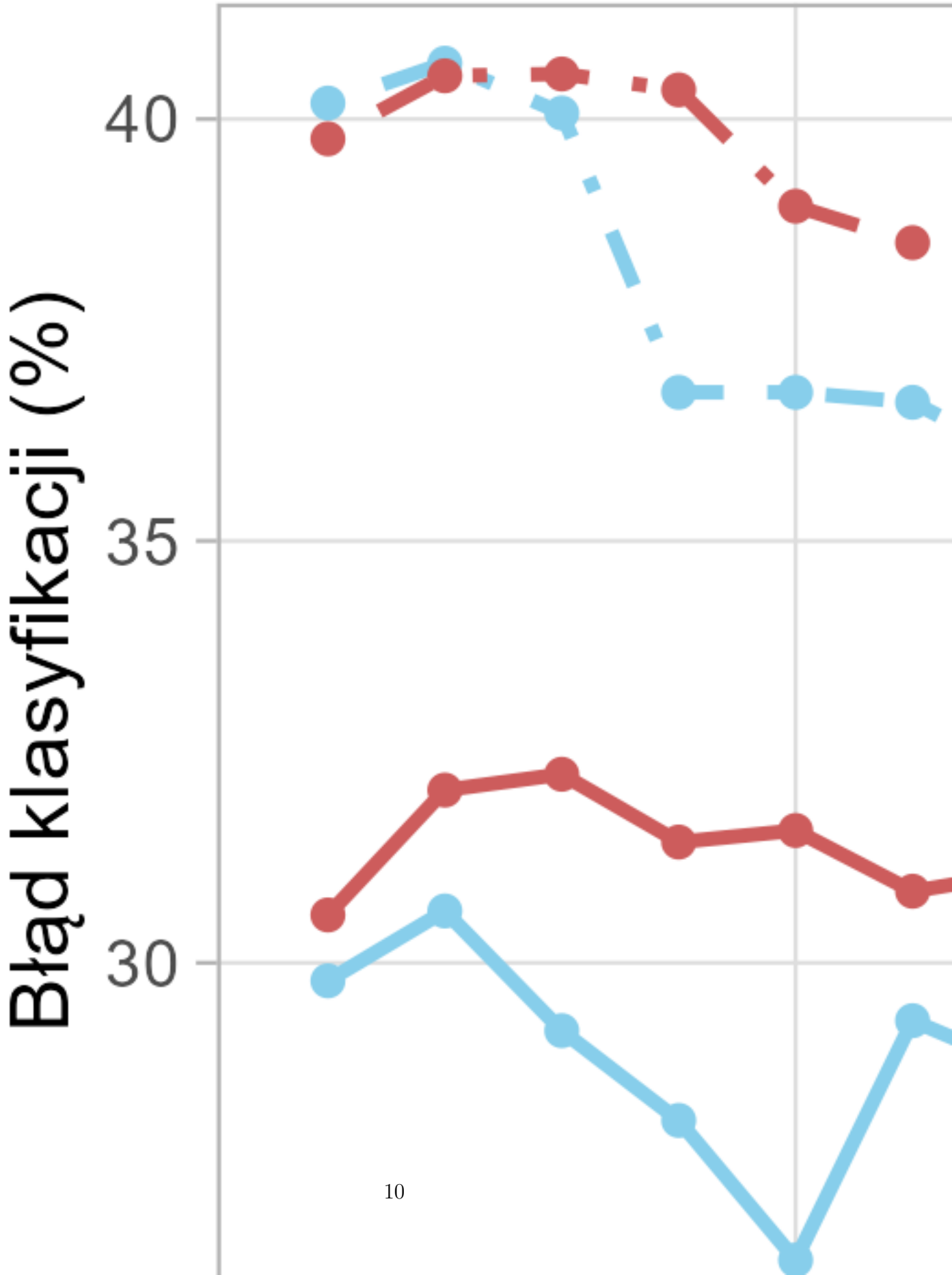
	bus	opel	saab	van
bus	65	7	10	2
opel	8	26	21	0
saab	0	20	28	1
van	0	18	16	60

Ograniczenie modelu do 3 cech spowodowało wzrost błędu o około 10 punktów procentowych. Analiza tabeli 4 wskazuje, że pogorszyła się zdolność rozróżniania klas `Opel` oraz `Saab`.

Co ciekawe, klasyfikacja pojazdów typu `Bus` oraz `Van` wciąż utrzymuje się na zadowalającym poziomie, co sugeruje, że cechy `Scat.Ra`, `Elong`, `D.Circ` niosą w sobie wystarczającą informację, pozwalającą na rozróżnianie klas wielkogabarytowych.

3.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*)

W tej sekcji zastosowano zaawansowane techniki estymacji błędu: 10-krotną walidację krzyżową (*10-fold Cross-validation*) oraz metodę *Bootstrap* (z 50 powtórzeniami). Obliczenia przeprowadzono na globalnie przeskalowanym zbiorze danych dla zakresu `k` od 1 do 20.


Błąd klasyfikacji (%)
40
35
30

Wykres 5 wskazuje na wyższą skuteczność modelu opartego na pełnym zbiorze 18 cech, który osiąga błędy o około 10 punktów procentowych niższe niż wariant uproszczony do 3 cech.

Dla pełnego zestawu zmiennych minimum błędu ($\approx 26.48\%$) przypada na niską wartość $\texttt{k} = 5$ przy zastosowaniu walidacji krzyżowej. Dla metody *Bootstrap* minimum błędu to $\approx 30.57\%$ dla $\texttt{k} = 1$.

W przypadku ograniczonej przestrzeni cech sytuacja się odwraca — obie metody dla małych wartości `k` generują najwyższy błąd ($\approx 40.66\%$), a najlepsze wartości są osiągane dla większych wartości ($\texttt{k} = 10$ dla *CV* oraz k = 12 dla *Bootstrap*).

Porównanie pokazuje, że *Bootstrap* (linia czerwona) generuje wyższe oceny błędu niż walidacja krzyżowa (linia niebieska). W przypadku modelu opartego o wszyskie zmienne, oba estymatory zaczynają zbiegać się ze sobą dopiero przy wartościach `k` powyżej 15.

3.3 Wnioski — metoda k-najbliższych sąsiadów

Tabela 5: Zestawienie wyników klasyfikacji k-NN

	Pełny zbiór (18 cech)	Wybrane zmienne (3 cechy)
k (pojedynczy podział)	9.00	14.00
k (CV)	5.00	10.00
k (Bootstrap)	1.00	12.00
Błąd - poj. podział (%)	27.66	36.52
Błąd - CV (%)	26.48	33.92
Błąd - Bootstrap (%)	30.57	36.46

W tabeli 5 zestawiono wyniki dokładności klasyfikacji różnych wariantów metody k-najbliższych-sąsiadów.

- Najwyższą dokładnością charakteryzuje się model oparty na pełnym zbiorze cech. Zaawansowane metody estymacji wskazują, że optymalna dla tego zbioru jest mała liczba sąsiadów: $\texttt{k} = 1$ lub $\texttt{k} = 5$.
- Redukcja przestrzeni do trzech zmiennych (`Scat.Ra`, `Elong`, `D.Circ`) upraszcza model obliczeniowo i pozwala zachować dobrą zdolność rozpoznawania pojazdów dużych (`Bus`, `Van`), lecz zwiększa błąd rozróżnienia samochodów osobowych (`Opel`, `Saab`).
- Zaawansowane schematy oceny dokładności potwierdziły, że wyniki uzyskane za pomocą pojedynczego podziału nie były przypadkowe — błędy z pojedynczego podziału są zbliżone do oszacowań uzyskanych metodami *Cross-validation* oraz *Bootstrap*.

4 Drzewa klasyfikacyjne (*classification trees*)

W niniejszym rozdziale przeprowadzono ocenę dokładności drzew klasyfikacyjnych przy użyciu pakietu `rpart`.

Analizę przeprowadzono w oparciu o różne zestawy parametrów (drzewa proste, złożone i przycięte) oraz dwa podzbiory zmiennych — pełny zestaw oraz cechy dobrze dyskryminujące (`Scat.Ra`, `Elong`, `D.Circ`).

Drzewa nie wymagają standaryzacji danych, dlatego operowano na oryginalnych zbiorach uczących i testowych.

4.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test).

4.1.1 Drzewa klasyfikacyjne dla pełnego oraz pomniejszonego zbioru zmiennych.

W pierwszej kolejności wyznaczono prognozy oraz błędy klasyfikacji na zbiorze uczącym i testowym dla drzew prostych (parametry domyślne) oraz drzew o celowo zwiększonej złożoności (`cp = 0.005`).

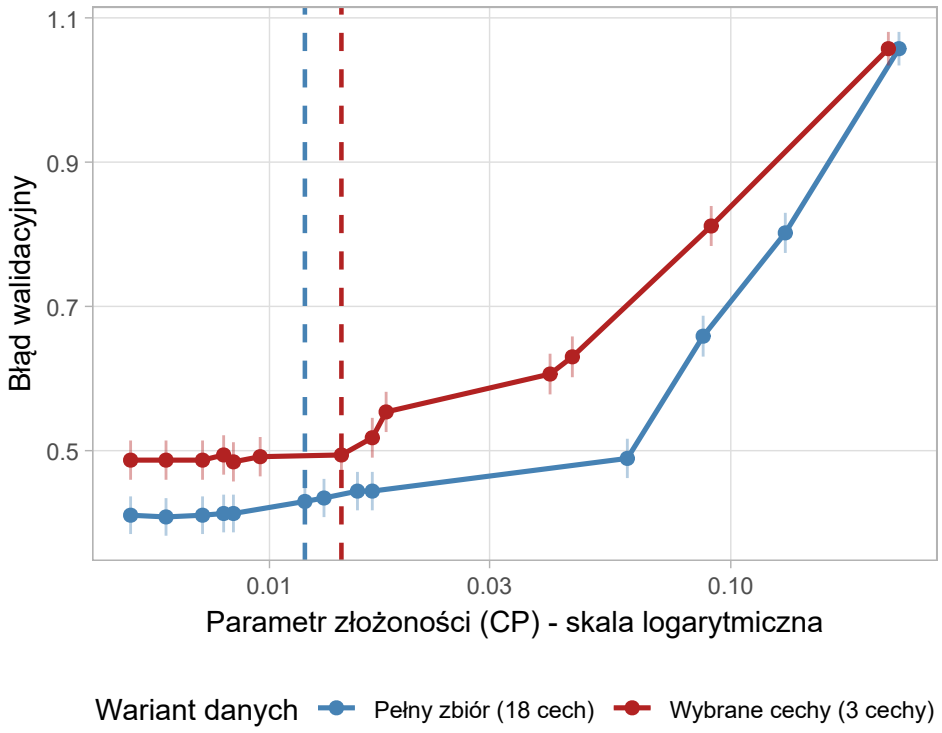
Uzyskane wartości błędów przedstawiają się następująco:

- Wariant pełny (18 cech): Drzewo proste wygenerowało błąd uczący 22.52% oraz testowy 35.82%. Dla drzewa złożonego błąd uczący spadł do 12.23%, a błąd testowy obniżył się do poziomu 31.56%.
- Wariant zredukowany (3 cechy): Drzewo proste osiągnęło błąd uczący 33.16% oraz testowy 40.07%. Wersja złożona zredukowała błąd uczący do 25.89%, pozostawiając błąd testowy na poziomie 39.72%.

Spadek błędu na zbiorze testowym po przejściu z parametrów domyślnych do konfiguracji złożonej w obu wariantach danych dowodzi, że bazowe ustawienia pakietu `rpart` nie były idealne.

W celu znalezienia złotego środka między prostotą a dokładnością, przeanalizowano parametr złożoności (`CP`). Parametr ten kontroluje proces przycinania drzewa decyzyjnego – większe wartości CP prowadzą do powstawania prostszych struktur, natomiast mniejsze wartości pozwalają na budowę bardziej rozbudowanych drzew.

Do wyboru ostatecznego modelu wykorzystano regułę jednego błędu standardowego (*1-SE*). Zgodnie z nią, spośród wszystkich analizowanych modeli wybiera się najprostsze drzewo, którego błąd walidacyjny nie przekracza minimalnego zaobserwowanego błędu powiększonego o jedno odchylenie standardowe. Przebieg tego procesu ilustruje wykres 6:

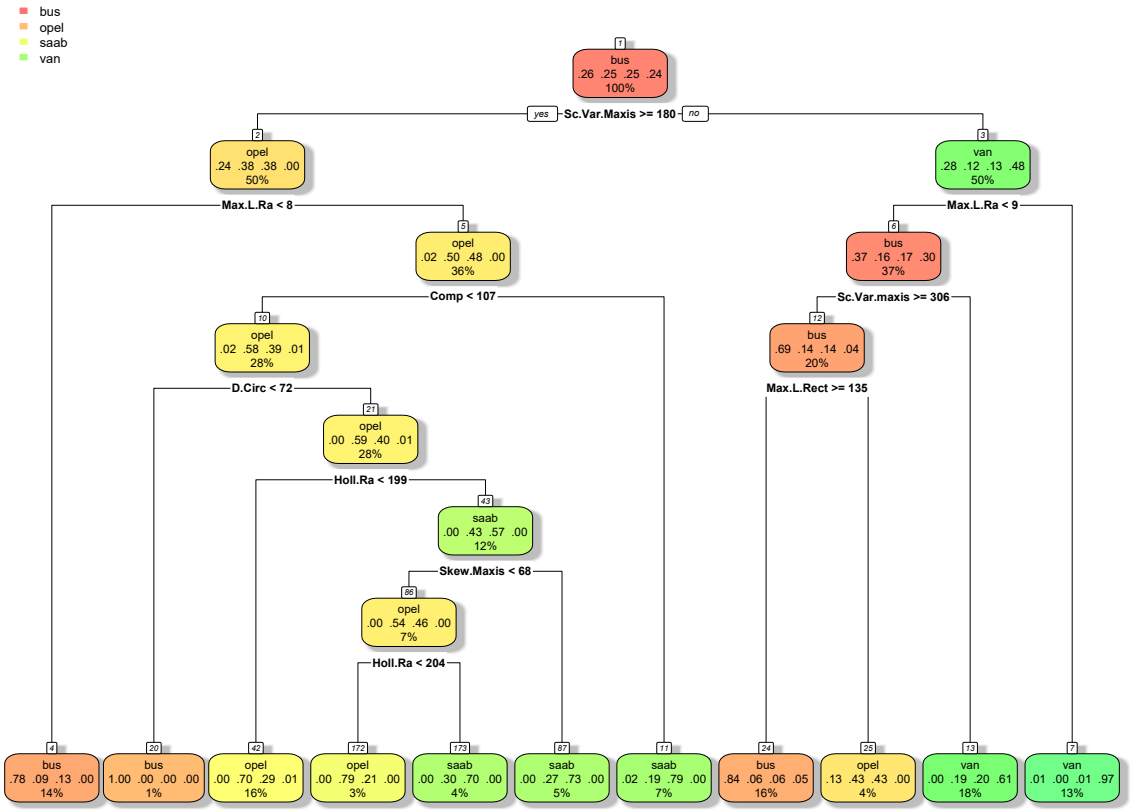


Rysunek 6: Porównanie błędu walidacji z zależności od parametru złożoności CP dla obu wariantów danych.

- Krzywa reprezentująca pełny zbiór 18 zmiennych (oznaczona kolorem niebieskim) na całej długości osi złożoności jest położona niżej niż krzywa dla 3 wybranych cech (kolor czerwony). Dowodzi to, że model zasilany kompletem informacji dysponuje znacznie większym potencjałem dobrej klasyfikacji.
- Zastosowanie reguły jednego błędu standardowego pozwoliło na wyznaczenie optymalnych punktów przycięcia, które na wykresie zostały zobrazowane liniami przerywanymi.

Reguła *1-SE* wyznaczyła optymalne parametry przycinania na poziomie 0.0119 dla pełnego modelu oraz 0.0143 dla zredukowanego.

Strukturę zoptymalizowanego drzewa dla pełnego zestawu cech przedstawiono na rysunku 7.



Rysunek 7: Wizualizacja struktury drzewa złożonego po operacji przycięcia.

Zdolności klasyfikacyjne ostatecznych, przyciętych modeli na danych testowych zweryfikowano za pomocą macierzy pomyłek (tabele 6 oraz 7).

Błąd klasyfikacji dla modelu opartego na 18 zmiennych ustabilizował się na poziomie 37.59%. Drzewo to doskonale radzi sobie z identyfikacją klas `Bus` oraz `Van`. Największym wyzwaniem pozostaje rozróżnienie samochodów osobowych marek `Opel` i `Saab`, gdzie model błędnie przypisał aż 23 pojazdów marki `Saab` jako `Opel`.

Tabela 6: Macierz pomyłek dla optymalnego (przyciętego) drzewa – pełny zbiór cech (zbiór testowy)

	bus	opel	saab	van
bus	65	9	9	2
opel	7	30	23	6
saab	0	16	26	0
van	1	16	17	55

Tabela 7: Macierz pomyłek dla optymalnego (przyciętego) drzewa – wybrane 3 cechy (zbiór testowy)

	bus	opel	saab	van
bus	60	8	9	2
opel	8	15	14	0
saab	3	28	32	0
van	2	20	20	61

Dla modelu zredukowanego błąd końcowy wzrósł do wartości 40.43%. Ograniczenie przestrzeni wejściowej do 3 cech pozbawiło algorytm krytycznych informacji, co — jak obrazuje tabela 7 — doprowadziło do osłabienia zdolności rozróżniania klas `Opel` oraz `Saab`, drastycznie nasilając losowe pomyłki między nimi.

4.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*)

Aby uniezależnić ocenę drzew klasyfikacyjnych od losowości pojedynczego podziału na zbiory uczące i testowe, przeprowadzono estymację błędu za pomocą 10-krotnej walidacji krzyżowej oraz metody *Bootstrap* (50 powtórzeń). W obliczeniach użyto drzewa zoptymalizowanego za pomocą wyznaczonych wcześniej parametrów kosztu-złożoności (`CP`).

Wyznaczony błąd klasyfikacyjny wyniósł:

- Dla modelu opartego na 18 zmiennych — 33.69% (CV), 32.19% (*Bootstrap*).
- Dla modelu uproszczonego — 38.42% (CV), 38.13% (*Bootstrap*).

4.3 Wnioski — drzewa klasyfikacyjne

Zbiorcze zestawienie wyników w tabeli 8 pozwala na wysunięcie wniosków:

Tabela 8: Zestawienie ocen błędów klasyfikacji dla zoptymalizowanych drzew

	Pełny zbiór (18 cech)	Wybrane cechy (3 cechy)
Pojedynczy podział [%]	37.59	40.43
Cross-validation [%]	33.69	38.42
Bootstrap [%]	32.19	38.13

- Wyniki uzyskiwane za pomocą prostego podziału na zbiór uczący i testowy okazały się stosunkowo (2–4 punkty procentowe) zbliżone do zaawansowanych metod testowania.
- Niezależnie od wybranego sposobu testowania, drzewa oparte na komplecie 18 cech za każdym razem radzą sobie lepiej od modeli celowo uproszczonych (różnica wynosi ok. 4–6 punktów procentowych błędu).
- Mimo optymalnego przycięcia drzew za pomocą reguły *1-SE*, natura popełnianych błędów (widoczna w macierzach pomyłek) pozostała niezmienna. Algorytm dobrze separuje klasy dużych pojazdów. Nierozwiązanym problemem pozostaje nakładanie się cech dla klas `Opel` i `Saab`.

5 Klasyfikator Naiwny Bayesa (*Naive Bayes*)

W niniejszym rozdziale sprawdzono skuteczność *Klasyfikatora Naiwnego Bayesa* przy użyciu pakietu `e1071`.

Podobnie jak w poprzednich krokach, osobno sprawdzono błędy klasyfikacji dla pełnego zestawu 18 zmiennych oraz zredukowanego podzbioru 3 cech (`Scat.Ra`, `Elong`, `D.Circ`).

W przypadku tej metody zrezygnowano ze wstępnej standaryzacji zmiennych. W przeciwieństwie do metod opartych na odległości geometrycznej, ten model analizuje każdą cechę niezależnie, estymując parametry funkcji gęstości osobno dla każdego wymiaru.

5.1 Ocena modeli na podstawie pojedynczego podziału (Train/Test)

5.1.1 Klasyfikator Naiwny Bayesa dla pełnego zbioru zmiennych

Utworzono model na postawie wszystkich zmiennych i wyznaczono błędy klasyfikacji na zbiorze testowym i uczącym:

- Błąd na zbiorze uczącym wyniósł 48.6%,
- Błąd na zbiorze testowym wyniósł: 56.4%.

W przypadku *Klasyfikatora Naiwnego Bayesa* błędy na zbiorze uczącym i testowym są zauważalnie większe w porównaniu do wcześniej analizowanych metod.

Tabela 9: Macierz pomyłek dla Klasyfikatora Naiwnego Bayesa – pełny zbiór cech

	bus	opel	saab	van
bus	11	1	1	5
opel	7	29	18	2
saab	8	14	29	2
van	47	27	27	54

Strukturę pomyłek dla modelu domyślnego na danych testowych przedstawia tabela 9. Analiza macierzy pomyłek ujawnia słabość tego algorytmu w kontekście zbioru `Vehicle`. Klasyfikator generuje wysoki błąd.

O ile pojazdy klasy `Van` zostały zadowalająco zklasyfikowane, o tyle samochody osobowe (`Opel` oraz `Saab`) oraz pojazdy typu `Bus` są nagminnie błędnie przypisywane do klasy `Van`. Wynika to bezpośrednio z „naiwnego" założenia o niezależności zmiennych.

5.1.2 Klasyfikator Naiwny Bayesa dla zmiennych o najlepszej zdolności dyskryminacyjnej

Identyczną procedurę powtórzono dla podzbioru 3 wybranych zmiennych.

Wyniki dla przestrzeni zredukowanej ukształtowały się następująco:

- Błąd na zbiorze uczącym wyniósł 57.6%,
- Błąd na zbiorze testowym wyniósł 65.6%.

Wbrew intuicji związanej z teoretycznymi właściwościami *Klasyfikatora Naiwnego Bayesa*, ograniczenie liczby zmiennych wyraźnie pogorszyło zdolności modelu. Błąd klasyfikacji wzrósł o około 7–9 punktów procentowych.

5.2 Zaawansowane schematy oceny dokładności (*Cross-validation* i *Bootstrap*)

W celu pełnego porównania skuteczności, przeprowadzono estymację błędu za pomocą 10-krotnej walidacji krzyżowej oraz metody *Bootstrap* (50 powtórzeń).

Dla modelu opartego o wszystkie zmienne estymowany błąd wyniósł 54.37% (CV) oraz 54.84% (*Bootstrap*). W przypadku uproszczonego modelu błąd wyniósł 61.23% (CV) oraz 61.67% (*Bootstrap*).

5.3 Wnioski - Klasyfikator Naiwny Bayesa

Tabela 10: Zestawienie ocen błędów klasyfikacji dla Klasyfikatora Naiwnego Bayesa

	Pełny zbiór (18 cech)	Wybrane cechy (3 cechy)
Pojedynczy podział [%]	56.38	65.60
Cross-validation [%]	54.37	61.23
Bootstrap [%]	54.84	61.67

Zestawienie uzyskanych wyników (tabela 10) pozwala na sformułowanie następujących wniosków:

- Ograniczenie liczby zmiennych z 18 do 3 pogorszyło zdolności modelu. Niezależnie od sposoby wyznaczenia błędy, wzrósł on o około 7–9 punktów procentowych — w przypadku walidacji krzyżowej z poziomu 54.37% do 61.23%, a dla pojedynczego podziału aż do 65.60%.

  Choć algorytm cierpi z powodu silnych zależności między cechami, to odrzucenie aż 15 zmiennych pozbawiło model zbyt dużej ilości informacji. Pozostałe 3 zmienne geometryczne okazały się niewystarczające, by poprawnie opisać i rozróżnić kształty pojazdów.

- Model zbudowany poprzez pojedynczy podział przyniósł najwyższe wartości błędu (56.38% dla pełnego zbioru i 65.60% dla zredukowanego). Walidacja krzyżowa oraz *Bootstrap* zwróciły wyniki wyraźnie niższe i bardziej optymistyczne (CV odpowiednio: 54.37% i 61.23%; Bootstrap: 54.84% i 61.67%).

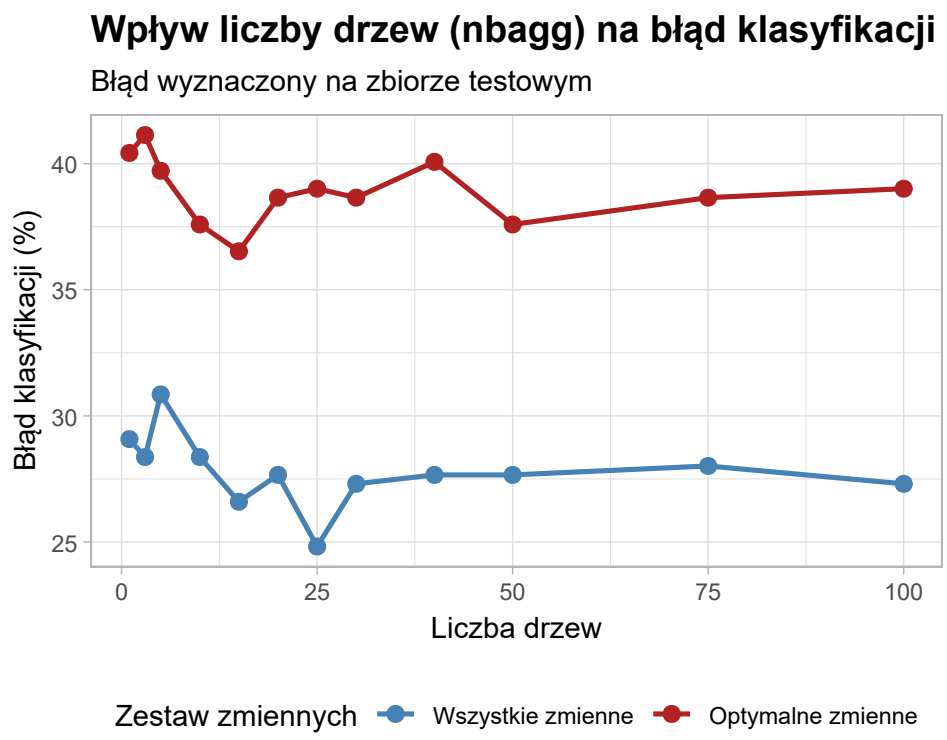
- Uzyskane wyniki (błędy przekraczające 54%) potwierdzają, że klasyfikator Naiwny Bayesa okazał się bardzo słabą metodą klasyfikacji w przypadku zbioru `Vehicle`

6 Algorytm *Bagging*

Algorytm *Bagging* to technika, która polega na jednoczesnym trenowaniu wielu niezależnych klasyfikatorów — najczęściej drzew decyzyjnych — na losowych próbkach pobranych ze zbioru uczącego.

6.1 Optymalizacja parametru `nbagg`

W celu znalezienia optymalnej liczby drzew decyzyjnych (parametr `nbagg`) sprawdzono wartości błędu klasyfikacji na zbiorze testowym dla obu rozważanych wariantów cech. Z uwagi na złożoność obliczeniową, testowano dla wartości nbagg $\in \{1, 3, 5, 10, 15, 20, 25, 30, 40, 50, 75, 100\}$.



Rysunek 8: Porównanie dokładności klasyfikacji algorytmu *Bagging* (dla zbioru testowego) w zależności od liczby drzew (parametr nbagg)

Przebieg błędu testowego w zależności od liczby drzew przedstawiono na wykresie 8. Widać wyraźną dysproporcję w dokładności klasyfikacji na korzyść modelu zasilanego pełnym zestawem cech (linia niebieska), który w badanym obszarze przedziale osiąga niższy błąd niż model oparty na trzech wybranych zmiennych (linia czerwona).

Dla pełnego zbioru danych minimum błędu testowego osiągnięto dla wartości `nbagg` = 25. W przypadku zbioru ograniczonego, optymalny błąd zanotowano dla `nbagg` = 15.

6.2 Analiza błędu klasyfikacji

Dla wyznaczonych parametrów przeprowadzono dokładniejsze obliczenia — zarówno dla pojedynczego podziału na zbiory testowy ($\frac{2}{3}$ danych) i uczący ($\frac{1}{3}$ danych), jak i przy użyciu 10-krotnej walidacji krzyżowej (CV) oraz metody *Bootstrap* (30 powtórzeń).

Wyniki można znaleźć w tabeli 11. Ich analiza pozwala na sformułowanie następujących wniosków:

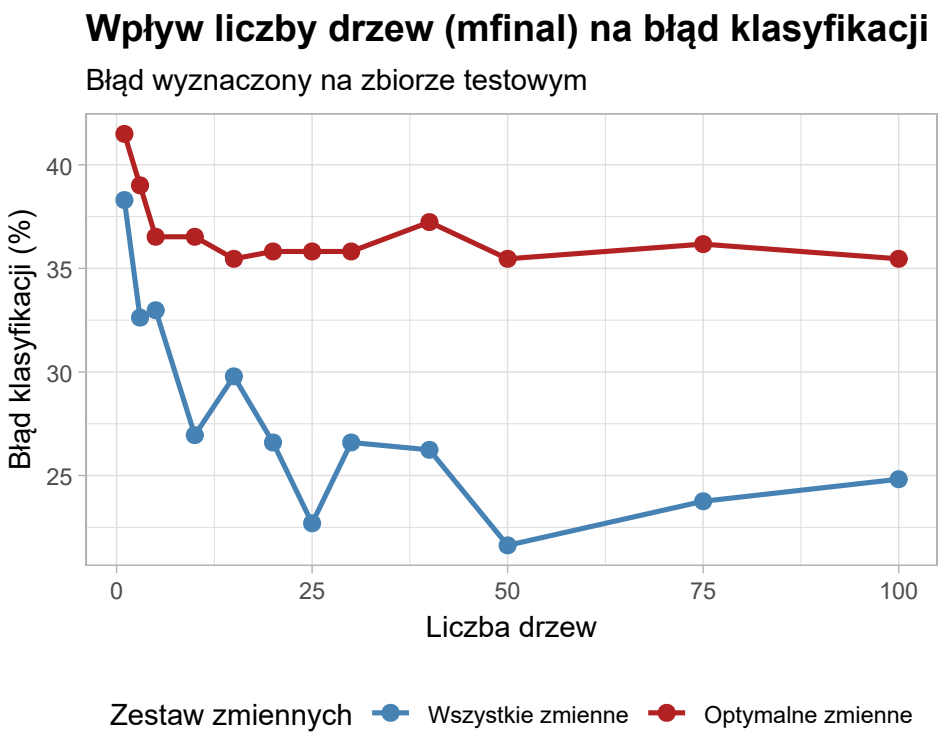
- Niezależnie od zastosowanej metody estymacji, model wyszkolony na pełnym zestawie 18 cech osiąga niższe wartości błędu (w przedziale 27.30% – 28.72%) w porównaniu do modelu opartego na 3 cechach (33.87% – 39.72%). Algorytm *Bagging* dobrze radzi sobie z większą liczbą zmiennych, a wcześniejsze ograniczenie zbioru doprowadziło do utraty istotnych informacji.
- Najniższe oceny błędu uzyskano dzięki 10-krotnej walidacji krzyżowej (27.30% dla pełnego zbioru). Estymator bootstrapowy dał wyniki nieznacznie wyższe (28.07% oraz 37.61%). W obu wariantach danych najwyższą wartość błędu wskazał pojedynczy podział na zbiór uczący i testowy (28.72% oraz 39.72%).

Tabela 11: Zestawienie ocen błędów klasyfikacji dla algorytmu bagging

	Pełny zbiór (18 cech)	Wybrane cechy (3 cechy)
Pojedynczy podział [%]	28.72	39.72
Cross-validation [%]	27.30	33.87
Bootstrap [%]	28.07	37.61

7 Algorytm *AdaBoost*

Algorytm *Boosting*, a w szczególności jego odmiana *AdaBoost*, reprezentuje odmienne podejście do budowy modeli niż *Bagging*. Zamiast równoległego i niezależnego trenowania drzew, *Boosting* generuje klasyfikatory sekwencyjnie.



Rysunek 9: Porównanie dokładności klasyfikacji algorytmu *AdaBoost* (dla zbioru testowego) w zależności od parametru mfinal

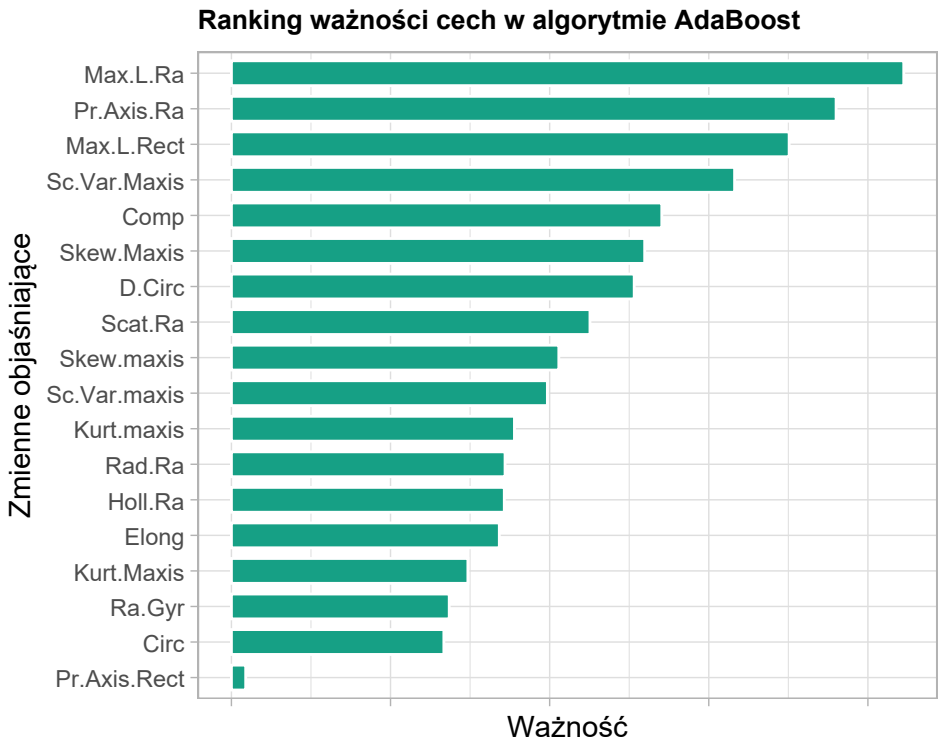
7.1 Optymalizacja parametru `mfinal`

W przypadku tego algorytmu kluczowym zadaniem optymalizacyjnym jest zbadanie, jak liczba drzew (parametr `mfinal`) wpływa na błąd na zbiorze testowym. W celu wyznaczenia optymalnej wartości tego parametru, przetestowano wartości błędu klasyfikacji dla zestawu wartości mfinal $\in \{1, 3, 5, 10, 15, 20, 25, 30, 40, 50, 75, 100\}$. Obliczenia zrealizowano zarówno dla pełnego zestawu cech, jak i dla zbioru ograniczonego do 3 zmiennych.

Przebieg błędu przedstawiono na wykresie 9. Podobnie jak w przypadku algorytmu *Bagging*, dla małych wartości `mfinal` obserwuje się niestabilność błędu, natomiast dla większych wartości błędy się stabilizują.

Dla pełnego zbioru danych (linia niebieska) minimum błędu testowego osiągnięto dla wartości `mfinal` $= 50$. Dla zbioru ograniczonego (linia czerwona) optymalną dokładność zanotowano przy wartości `mfinal` $= 15$.

7.2 Ranking ważności cech w algorytmie *AdaBoost*



Rysunek 10: Zestawienie względnej ważności cech w *AdaBoost*

Jedną z zalet algorytmu *AdaBoost* jest możliwość wyznaczenia względnej ważności cech objaśniających (wykres 10). Co istotne, ranking ważności wygenerowany przez *AdaBoost* różni się od wstępnej selekcji opartej na estymatorach gęstości. Zgodnie z nim, wyznaczono nowy zbiór, zawierający jedynie cechy: `Max.L.Ra`, `Pr.Axis.Ra`, `Max.L.Rect`.

7.3 Analiza błędu klasyfikacji

Dla wyznaczonych optymalnych wartości parametru `mfinal` przeprowadzono ocenę zdolności klasyfikacyjnych algorytmu. Porównaniu poddano trzy warianty danych: pełny zbiór (18 cech), wybrane cechy początkowe oraz zestaw trzech cech optymalnych, wytypowanych bezpośrednio z wykresu 10. Wyniki zestawiono w tabeli 12.

Tabela 12: Zestawienie ocen błędów klasyfikacji dla algorytmu AdaBoost

	Pełny zbiór (18 cech)	Wybrane cechy (wg estymatorów gęstości)	Optymalne cechy (wg ważności)
Pojedynczy podział [%]	24.11	39.36	47.52
Cross-validation [%]	21.63	35.82	46.45
Bootstrap [%]	24.70	34.88	46.30

- Niezależnie od zastosowanej metody szacowania błędu, model zasilany kompletem 18 zmiennych osiąga zdecydowanie najniższy błąd klasyfikacji, który utrzymuje się na stabilnym poziomie (24.11% dla pojedynczego podziału, 21.63% dla *CV* oraz 24.70% dla *Bootstrap*).
- Ciekawe jest porównanie modeli opartych na zredukowanej przestrzeni cech. W każdym przypadku model oparty na estymatorach gęstości okazał się skuteczniejszy niż ten zbudowany na cechach z wykresu ważności. (pojedynczy podział (39.36% vs 47.52%), 10-krotna walidacja krzyżowa (35.85% vs 46.45%), *Bootstrap* (34.88% vs 46.30% dla gęstości).

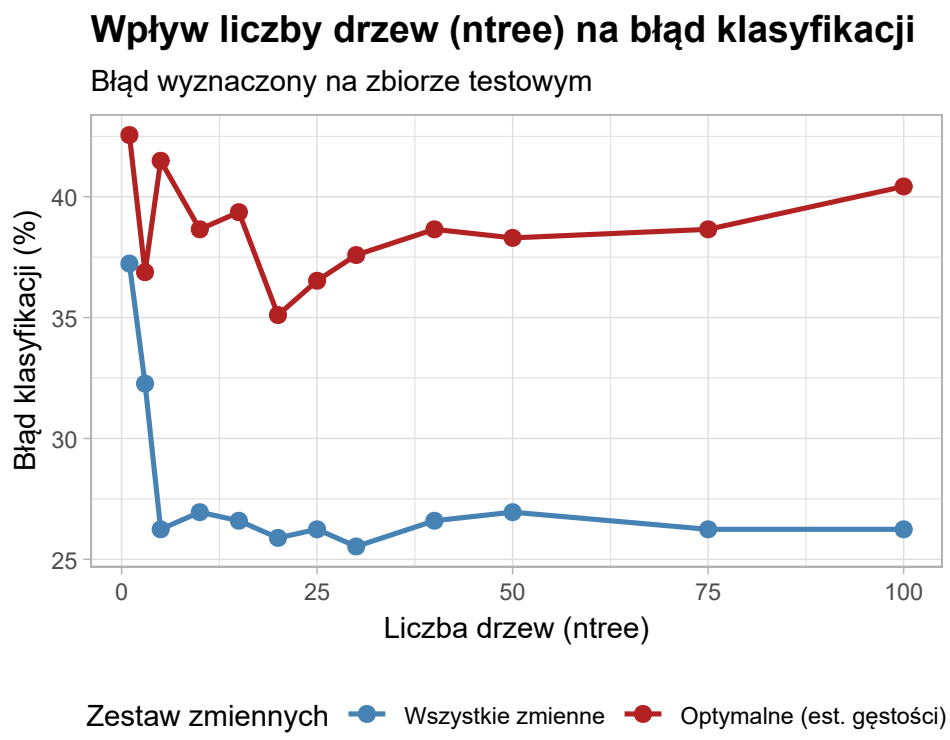
  Zjawisko to wynika ze specyfiki działania *AdaBoost*: wysoka ważność zmiennej w pełnym modelu oznacza, że była ona kluczowa w obecności pozostałych 15 cech. Jednak po odosobnieniu tych 3 zmiennych, utraciły one swoje właściwości. Z kolei estymator gęstości badał cechy pod kątem ich samodzielnej zdolności separacji klas, co sprawiło, że jako odizolowany zbiór sprawdziły się lepiej.

8 Lasy losowe

Lasy losowe (*Random Forest*) stanowią rozwinięcie idei algorytmu *Bagging*. Lasy wprowadzają jednak kluczową modyfikację mającą na celu zmniejszenie powiązania budowanych klasyfikatorów. W każdym węźle każdego drzewa proces poszukiwania optymalnego podziału nie odbywa się w oparciu o pełną przestrzeń cech, lecz na losowo wybranym podzbiorze zmiennych (parametr `mtry`, domyślnie równy pierwiastkowi z liczby wszystkich cech).

8.1 Optymalizacja parametru `ntree`

W kontekście optymalizacji tego modelu kluczowe jest określenie optymalnej wielkości lasu za pomocą liczby drzew (parametr `ntree`).



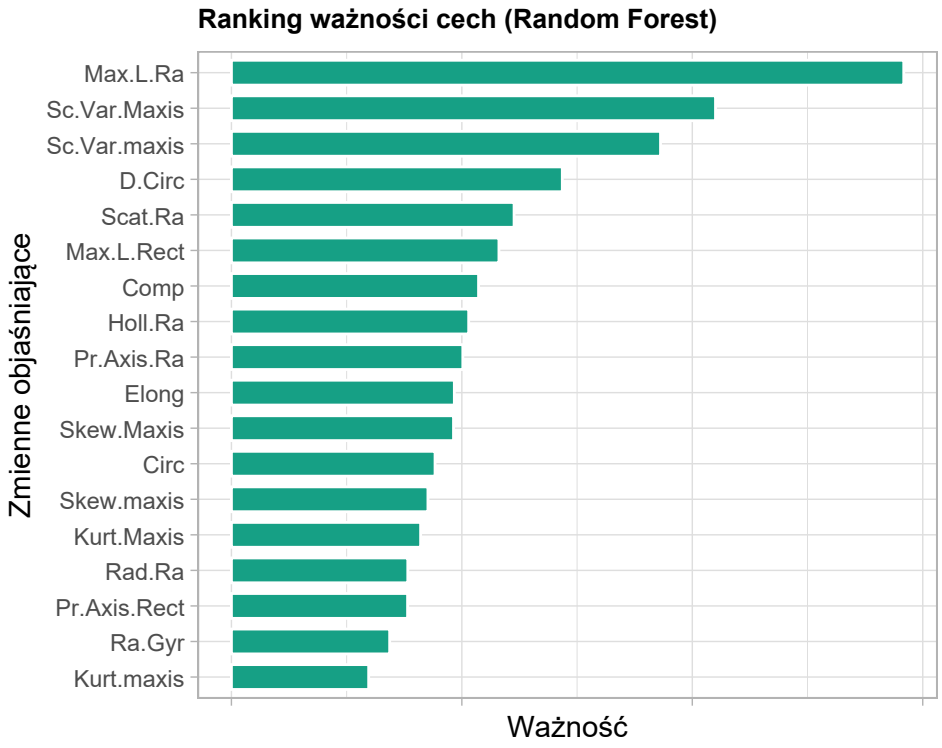
Rysunek 11: Porównanie dokładności klasyfikacji algorytmu *Random Forest* (dla zbioru testowego) w zależności od liczby drzew (parametr ntree)

W celu odnalezienia optymalnej wartości `ntree`, zbadano wartości błędu klasyfikacji na zbiorze testowym w zależności od liczby wygenerowanych drzew decyzyjnych. Obliczenia przeprowadzono dla wartości ntree $\in \{1, 3, 5, 10, 15, 20, 25, 30, 40, 50, 75, 100\}$, analizując równolegle model zbudowany na pełnym zestawie cech oraz podzbiorze 3 zmiennych dobrze dyskryminujących.

Przebieg błędu testowego w zależności od liczby drzew przedstawiono na wykresie 11. Dla małej liczby drzew (przedział ntree < 20) obie linie wykazują silne wahania, dla większych wartości się stabilizują. Po przekroczeniu granicy około 40–50 drzew obie krzywe błędu wyraźnie się wygładzają

Dla pełnego zestawu 18 zmiennych (linia niebieska) minimum błędu testowego osiągnięto dla wartości `ntree` = 30. W przypadku zbioru ograniczonego (linia czerwona) optymalny punkt pracy algorytmu odnotowano dla wartości `ntree` = 20.

8.2 Ranking ważności cech w algorytmie *Random Forest*



Rysunek 12: Zestawienie względnej ważności cech w metodzie *Random Forest*

Algorytm *Random Forest* również umożliwia wyznaczenie względnej ważności cech objaśniających. Na wykresie 12 przedstawiono ten ranking. Stworzono dedykowany podzbior cech optymalnych dla *Random Forest*, zawierający trzy najlepsze zmienne z tego wykresu — `Max.L.Ra`, `Sc.Var.Maxis`, `Sc.Var.maxis`. Wybór ten pozwala sprawdzić, czy w tym przypadku cechy wybrane przez algorytm radzą sobie lepiej w zredukowanej przestrzeni niż cechy

wyselekcjonowane za pomocą estymatorów gęstości.

8.3 Analiza błędu klasyfikacji

Tabela 13: Zestawienie ocen błędów klasyfikacji dla algorytmu Random Forest

	Pełny zbiór (18 cech)	Wybrane cechy (wg estymatorów gęstości)	Optymalne cechy (wg ważności)
Pojedynczy podział [%]	26.95	39.72	36.17
Cross-validation [%]	26.24	32.27	29.79
Bootstrap [%]	25.94	34.00	33.20

Dla wyznaczonych optymalnych parametrów wielkości lasu (`ntree`) przeprowadzono ocenę zdolności klasyfikacyjnych algorytmu. Porównaniu poddano trzy konfiguracje danych: pełny zbiór (18 cech), wybrane cechy początkowe (3 cechy wg estymatorów gęstości) oraz zestaw 3 cech optymalnych dla lasu losowego (wytypowanych na podstawie wykresu 12). Wyniki zestawiono w tabeli 13.

- Podobnie jak w przypadku poprzednich algorytmów, model zasilany kompletem 18 zmiennych uzyskuje najlepsze wyniki, osiągając błąd na poziomie 26.95% dla pojedynczego podziału, 26.24% dla *CV* oraz 25.94% dla *Bootstrap*.
- W przeciwieństwie do algorytmu *AdaBoost*, w przypadku lasu losowego trzy cechy wytypowane bezpośrednio przez ranking ważności okazały się lepsze we wszystkich scenariuszach od cech wybranych na podstawie estymatorów gęstości. W pojedynczym podziale błąd spadł z 39.72% do 36.17% (poprawa o blisko 3.5 punkta procentowego), w walidacji krzyżowej odnotowano spadek z 32.27% do 29.79%, natomiast w metodzie *Bootstrap* błąd zmalał z 34.00% do 33.20%.
- W analizowanym przypadku pojedynczy podział danych okazał się bardziej podatny na wpływ losowego rozdzielenia na zbiory uczący i testowy. Walidacja krzyżowa dostarczała najbardziej stabilnych i jednocześnie najbardziej optymistycznych ocen błędu. Natomiast metoda *Bootstrap*, zgodnie z jej charakterystyką, zwracała nieco wyższe oszacowania błędu.

8.4 Porównanie metod zespołowych z pojedynczym drzewem decyzyjnym

W celu jednoznacznej oceny zysku, jaki niesie ze sobą stosowanie zaawansowanych algorytmów klasyfikacji, przeprowadzono bezpośrednie porównanie uzyskanych wyników z modelem bazowym — pojedynczym drzewem decyzyjnym (omawianym na poprzednim sprawozdaniu). Jako kryterium porównawcze wybrano oceny błędu szacowane za pomocą metody *Bootstrap*. Porównano ze sobą dwie konfiguracje danych — pełny (18) oraz zredukowany (3) zbiór zmiennych.

Istotnym elementem analizy jest wskaźnik względnej redukcji błędu klasyfikacji (R), obliczany według wzoru:

$$R = \frac{\text{błąd}_{\text{poj. drzewo}} - \text{błąd}_{\text{algorytm}}}{\text{błąd}_{\text{poj. drzewo}}} \times 100\%$$

*__Uwaga__: W przypadku algorytmów *AdaBoost* oraz *Random Forest* w kolumnie dotyczącej zbioru zredukowanego przedstawiono wyniki uzyskane dla podzbioru zmiennych, który zapewnił niższy błąd klasyfikacji spośród dwóch analizowanych podejść: podzbiorów wyznaczonych na podstawie wykresów ważności zmiennych oraz podzbiorów wyznaczonych z wykorzystaniem estymatorów gęstości.

Zestawienie wartości błędów oraz wyznaczonego `R` przedstawiono w tabeli 14.

Tabela 14: Zestawienie ocen błędu (Bootstrap) oraz względnej redukcji błędu w stosunku do pojedynczego drzewa

Model klasyfikacyjny	Pełny zbiór (18 cech) [%]	Względna redukcja błędu (R)	Zbiór zredukowany (3 cechy)* [%]	Względna redukcja błędu (R)
Pojedyncze drzewo	32.19	—	38.13	—
Bagging	28.07	12.80%	37.61	1.36%
AdaBoost	24.70	23.27%	34.88	8.52%
Random Forest	25.94	19.41%	33.20	13.01%

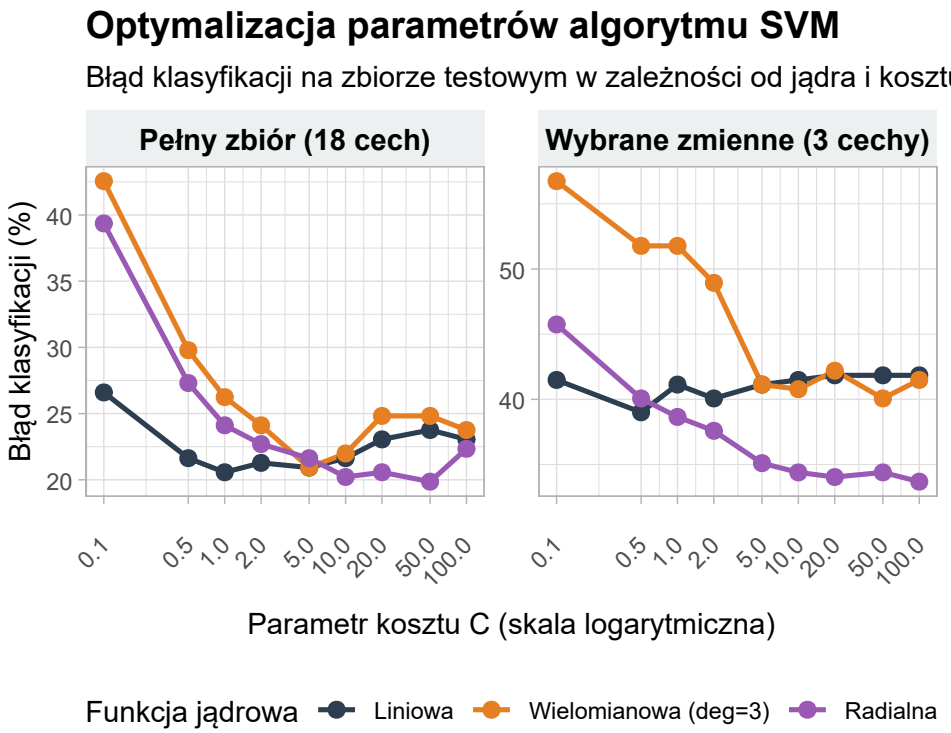
- Wszystkie trzy algorytmy (*Bagging*, *AdaBoost*, *Random Forest*) okazały się lepsze od pojedynczego drzewa decyzyjnego, niezależnie od wymiarowości przestrzeni cech.
- W przypadku zasilania modeli kompletem 18 cech, bezkonkurencyjny okazał się algorytm *AdaBoost*, redukując błąd bazowy o blisko jedną czwartą (23.27% względnego zysku). Drugie miejsce zajął las losowy (19.41% redukcji błędu).
- Sytuacja ulega zmianie w przestrzeni zredukowanej do 3 cech. Tutaj liderem jest *Random Forest*, który osiągnął najniższy błąd (33.20%) i najwyższą względną redukcję błędu (13.01%).
- Algorytm *Bagging*, choć na pełnym zbiorze radzi sobie dobrze (12.80% zysku), na zbiorze zredukowanym przyniósł symboliczny zysk (zaledwie 1.36% względnej redukcji).

9 Metoda wektorów nośnych (SVM)

Metoda wektorów nośnych (*SVM*) jest deterministyczną metodą klasyfikacji, której działanie opiera się na wyznaczeniu hiperpłaszczyzny maksymalizującej odległości (marginesu) pomiędzy klasami.

9.1 Optymalizacja parametrów `C` oraz `Kernel`

Na skuteczność klasyfikacji metodą SVM istotny wpływ ma kilka parametrów.



Rysunek 13: Wpływ funkcji jądrowej oraz parametru C na błąd klasyfikacji algorytmu SVM

Pierwszym z nich jest parametr `kernel`, określający rodzaj funkcji jądrowej wykorzystywanej do transformacji danych do przestrzeni o wyższym wymiarze.

Drugim istotnym parametrem jest parametr kosztu (`C`), który określa kompromis pomiędzy maksymalizacją marginesu a liczbą błędów klasyfikacji w zbiorze uczącym. Niskie wartości parametru dopuszczają większą liczbę błędnie sklasyfikowanych obserwacji, prowadząc do modelu o większej odporności na szum i niższej wariancji. Z kolei wysokie wartości powodują silniejsze karanie błędów klasyfikacji, co sprzyja dokładnemu dopasowaniu modelu do danych uczących, lecz jednocześnie zwiększa ryzyko przeuczenia.

W przypadku jądra radialnego dodatkowym parametrem jest (γ), który określa zasięg wpływu pojedynczych obserwacji na kształt granicy decyzyjnej. Małe wartości prowadzą do gładkiej granicy decyzyjnej i bardziej uogólnionego modelu, natomiast duże wartości powodują, że wpływ pojedynczych obserwacji jest ograniczony do niewielkiego obszaru, co umożliwia tworzenie bardziej złożonych granic decyzyjnych, ale jednocześnie zwiększa ryzyko przeuczenia. W używanym pakiecie `e1071` domyślna wartość parametru γ wynosi $(1/p)$, gdzie (p) oznacza liczbę zmiennych objaśniających w zbiorze danych. W tej sekcji użyto domyślnej γ.

Wykres 13 dostarcza wniosków na temat zachowania błędu klasyfikacji w zależności od wymiaru zbioru danych oraz parametrów:

Zasilenie algorytmu pełnym zestawem zmiennych pozwala na uzyskanie niskich poziomów błędu (schodzących poniżej granicy 25%), jednak dynamika poszczególnych jąder się różni:

- Jądro liniowe (najskuteczniejsze dla małych wartości C): Dla skrajnie małego $C = 0.1$ marginesu jądro liniowe deklasuje konkurencję, osiągając błąd na poziomie ok. 26%. Swoje globalne minimum osiąga dla $C = 1$ (ok. 20.5%). Co istotne, dalsze zwiększanie `C` powoduje wzrost błędu aż do ok. 24% dla $C = 50$. W 18-wymiarowej przestrzeni zbyt restrykcyjne dążenie do bezbłędnego rozdzielenia próby uczącej doprowadziło do utraty zdolności generalizacji.

- Jądro wielomianowe (stopnia 3): Wykazuje dla $C = 0.1$ generuje błąd rzędu 42%. Krzywa błędu spada jednak najgwałtowniej, osiągając optimum przy $C = 5.0$ (ok. 21%), po czym jej jakość ponownie się pogarsza.

- Jądro radialne, podobnie jak wielomianowe, zaczyna wysoko (ok. 40% dla $C = 0.1$), lecz wykazuje najbardziej stabilny spadkek. Jądro osiąga globalne minimum całego eksperymentu dla $C = 50.0$ (ok. 19.8%). Dopiero przy $C = 100.0$ błąd zaczyna nieznacznie wzrastać.

Redukcja przestrzeni informacyjnej do zaledwie 3 zmiennych spowodowała przesunięcie całego pasma błędu w górę (w przedział od 35% do ponad 55%), zmieniając hierarchię efektywności funkcji jądrowych:

- Jądro radialne, w przeciwieństwie do pozostałych konfiguracji, jako jedyne potrafiło podjąć skuteczną walkę z ograniczoną przestrzenią cech. Rozpoczynając od błędu ok. 45% ($C = 0.1$), wykazuje stopniową poprawę wraz ze wzrostem kosztu, osiągając swoje optimum dla $C = 100.0$ z wynikiem ok. 35%.

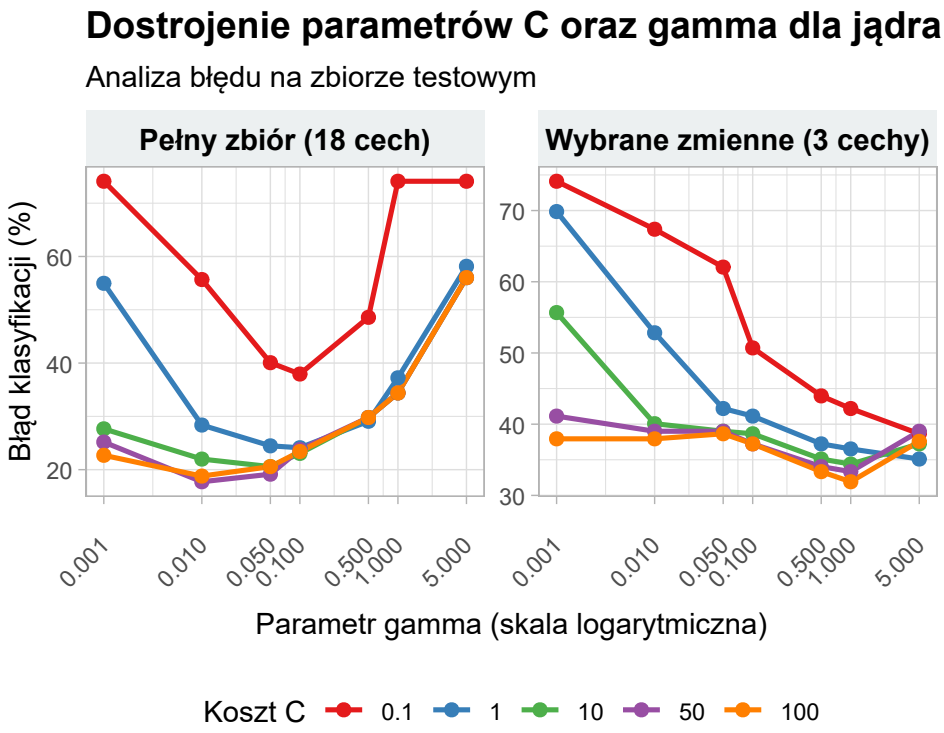
- Jądro liniowe wykazało całkowitą niewydolność. Przebieg krzywej jest niemal płaski i oscyluje wokół bardzo wysokiego błędu 39%–42%.

- Jądro wielomianowe (stopnia 3), podobnie jak na pełnym zbiorze, startuje z fatalnego poziomu (ok. 55% błędu), po czym stabilizuje się na poziomie ok. 41% dla $C \geq 5.0$, oferując tylko minimalny zysk względem prostszego jądra liniowego.

Na podstawie uzyskanych wyników do dalszych etapów analizy przyjęto konfiguracje modelu zapewniające najniższą wartość błędu. Dla pełnego zbioru danych zastosowano jądro radialne oraz parametr kosztu $C = 50$. Natomiast dla zredukowanego zbioru wybrano jądro radialne (RBF) z parametrem kosztu $C = 100$.

9.2 Optymalizacja parametru `gamma` dla jądra radialnego

W przypadku jądra radialnego kluczem do wykorzystania pełnego potencjału *SVM* jest jednoczesne zbalansowanie parametru kosztu C oraz szerokości jądra γ. Wyniki poszukiwań optymalnej kombinacji tych parametrów zaprezentowano na wykresie 14.



Rysunek 14: Zależność parametrów C oraz gamma i ich wpływ na błąd klasyfikacji jądra radialnego

- **Pełny zbiór danych:** Dla niskich γ oraz niskiego kosztu $C = 0.1$ błąd wynosi aż 75%, a dla najwyższych wartości C oscyluje wokół 23–27%. Punkt optymalny dla pełnego zbioru danych (który jest globalnym minimum) zachodzi dla $\gamma = 0.010$, gdzie najlepsze rezultaty osiągają modele o wysokim koszcie: $C = 50$ (błąd ok. 17.5%) oraz $C = 100$ (błąd ok. 18.5%). Świadczy to o tym, że w przestrzeni 18 cech optymalna granica jest relatywnie łagodna (niska wartość γ), ale wymaga dużego C, który surowo karze pomyłki na zbiorze uczącym.

  Dla wysokich $\gamma > 0.1$): następuje dość gwałtowny wzrost błędu dla wszystkich wartości kosztu. Dla $C = 0.1$ oraz błąd gwałtownie osiąga krytyczny pułap 75%, a dla pozostałych linii (w tym $C = 50$ i $C = 100$) błąd szybuje w okolice 55–58%.

- **Zawężony zbiór danych:** Następuje tutaj odwrócenie trendu w stosunku do pełnego zbioru zmiennych. Zwiększanie parametru γ od wartości 0.001 aż do 1.000 skutkuje monotonicznym spadkiem błędu dla każdej testowanej wartości kosztu C. Dopiero przy skrajnej wartości $\gamma = 5$ błąd zaczyna ponownie wzrastać dla wyższych wartości kosztu. Najlepsze wyniki osiągane są w $\gamma = 1$ dla dla $C = 100$ (błąd ok. 32%).

W oparciu o zaobserwowane wyniki, w końcowej tabeli porównawczej uwzględniono wyniki algorytmu *SVM* dla następujących parametrów optymalnych:
Dla pełnego zbioru (18 cech) — jądro radialne, $C = 50$, $\gamma = 0.010$.
Dla zbioru zredukowanego (3 cechy) — jądro radialne, $C = 100$, $\gamma = 1.000$.

9.3 Analiza błędu klasyfikacji

Tabela 15: Wpływu dostrojenia parametru gamma na pełnym zbiorze (18 cech)

Metoda estymacji błędu	Optymalny koszt (C=50, gamma domyślna)	Pełna optymalizacja (C=50, gamma=0.01)
Pojedynczy podział [%]	19.86	17.73
Cross-validation [%]	15.96	17.73
Bootstrap [%]	19.90	18.44

W tabeli 15 przedstawiono wyniki dla pełnego zbioru zmiennych.

Na pierwszy rzut oka model z domyślną gammą i kosztem $C = 50$ osiągnął spektakularny, najniższy błąd w walidacji krzyżowej (15.96%). Jednak te same parametry na pojedynczym podziale (19.86%) oraz w metodzie *Bootstrap* (19.90%) radzą sobie zauważalnie gorzej. Świadczy to o niestabilności modelu — domyślna gamma tworzyła granice decyzyjne, które szczęściliw w niektórych podziałach *CV* dawały świetny wynik.

Ręczne obniżenie parametru γ do poziomu 0.01 spowodowało, że wyniki stały się stabilne i spójne: pojedynczy podział (17.73%), *Cross-validation* (17.73%) oraz *Bootstrap* (18.44%).

Choć straciliśmy 1.77 punktu procentowego w *CV*, pełna optymalizacja dostarczyła model bardziej powtarzalny.

Tabela 16: Wpływu dostrojenia parametru gamma na zbiorze zredukowanym (3 cechy)

Metoda estymacji błędu	Optymalny koszt (C=100, gamma domyślna)	Pełna optymalizacja (C=100, gamma=1)
Pojedynczy podział [%]	33.69	31.91
Cross-validation [%]	30.50	30.85
Bootstrap [%]	32.41	32.97

W tabeli 16 przedstawiono wyniki dla zredukowanego zbioru zmiennych (3 cechy). Wyniki modeli z domyślną gammą oraz zoptymalizowaną ($\gamma = 1$) okazały się bardzo zbliżone.

Pełna optymalizacja przyniosła lekką poprawę w pojedynczym podziale (spadek z 33.69% do 31.91%), podczas gdy błędy *CV* (30.50% vs 30.85%) oraz *Bootstrap* (32.41% vs 32.97%) wskazują minimalnie na korzyść domyślnej gammy.

Przeprowadzone obliczenia pokazują, że optymalizacja parametru γ ma kluczowe znaczenie w przestrzeniach wielowymiarowych (18 cech). W przestrzeniach o niskiej wymiarowości (3 cechy) rola parametru γ maleje.

10 Podsumowanie wyników

Tabela 17: Globalne podsumowanie błędu Bootstrap dla wszystkich badanych metod klasyfikacji

Algorytm/Model	Pełny zbiór (18 cech) [%]	Zbiór zredukowany (3 cechy) [%]
k-Najbliższych Sąsiadów (k-NN)	30.57	36.46
Pojedyncze drzewo decyzyjne	32.19	38.13
Naiwny Bayes	54.84	61.67
Bagging	28.07	37.61
Random Forest	25.94	33.20
AdaBoost	24.70	34.88
SVM	**18.44**	**32.41**

W celu ostatecznej oceny wszystkich omawianych metod klasyfikacji (również z listy 3), zestawieno oceny błędu uzyskanych za pomocą metody *Bootstrap* (tabela 17).

- **Naiwny Bayes — 54.84%** Algorytm ten zanotował zdecydowanie najgorszy wynik, ocierający się o losowe zgadywanie. Jest to bezpośredni dowód na to, że cechy opisujące sylwetki pojazdów są ze sobą skorelowane. Naruszenie fundamentalnego założenia metody o niezależności cech całkowicie pozbawiło ten model zdolności predykcyjnych.
- **Drzewa pojedyncze i lasy — 32.19%** Pojedyncze drzewa radzą sobie przeciętnie z uwagi na skłonność do przeuczania i tworzenie sztywnych granic decyzyjnych. Przejście

do algorytmów zaawansowanych przynosi systematyczną poprawę: *Bagging* redukuje błąd do 28.07%, *Random Forest* osiąga 25.94%, a sekwencyjny *AdaBoost* wyciska z tej architektury maksimum, schodząc do poziomu 24.70%.

- **k-NN — 30.57%** Algorytm k-NN okazał się skuteczniejszy od pojedynczego drzewa decyzyjnego. Oznacza to, że podobieństwo euklidesowe między obserwacjami opisanymi przez 18 cech dobrze wspiera proces klasyfikacji. Jednocześnie najlepsze wyniki nadal uzyskały metody uczenia zespołowego.
- **SVM — 18.44%** Metoda Wektorów Nośnych z zoptymalizowanym jądrem radialnym zdeklasowała rywali. Wynik 18.44% jest o ponad 6 punktów procentowych lepszy od najlepszego algorytmu bazującego na drzewach (*AdaBoost*) i o ponad 12 punktów procentowych lepszy od *k-NN*.

Ograniczenie przestrzeni do 3 cech (prawa kolumna tabeli) wywołało gwałtowny, systematyczny wzrost błędu dla każdego badanego algorytmu, co potwierdza krytyczną utratę informacji o geometrii pojazdów.

- *SVM* utrzymuje pozycję lidera. Mimo deficytu informacji, w pełni zoptymalizowany model *SVM* obronił pierwsze miejsce na podium, osiągając najniższy błąd równy 32.41%.
- Przetasowanie w metodach uczenia zespołowego: W małej przestrzeni 3 cech algorytm *Random Forest* (33.20%) okazał się skuteczniejszy niż *AdaBoost* (34.88%). Dla zbioru zredukowanego las losowy okazał się skuteczniejszy od boostingu. Wyniki wskazują, że losowy dobór cech lepiej ograniczał ryzyko przeuczenia, podczas gdy boosting wykazywał większą skłonność do nadmiernego dopasowania modelu do danych uczących.
- *k-NN* (36.46%), *Bagging* (37.61%) oraz pojedyncze drzewo (38.13%) osiągnęły bardzo zbliżone, wysokie poziomy błędu. *Naiwny Bayes* utrzymał status najsłabszego modelu, pogarszając swój wynik aż do poziomu 61.67%.

Zbiór danych `Vehicle` wymaga stosowania metod odpornych na silne współzależności liniowe i nieliniowe zmiennych objaśniających (jak *SVM* lub *Random Forest*). Algorytmy opierające się na założeniu o niezależności (*Naiwny Bayes*) są w tym scenariuszu bezużyteczne.

Dla zredukowanego wymiaru cech do 3 zmiennych żaden algorytm nie jest w stanie zejść poniżej granicy ok. 32% błędu. Świadczy to o tym, że ekstrakcja cech usunęła kluczowe informacje różnicujące klasy.